AIMERZ.ai
Aim.Act.Achieve

NOTES

Getting Started with Java (Fundamentals and Setup)

Introduction to Java

Java is a popular and widely-used programming language. It was created by James Gosling at Sun Microsystems in 1995. Java is designed to be simple, secure, and versatile, allowing programmers to write code that can run on different devices without modification. It follows the Object-Oriented Programming (OOP) paradigm, meaning it organizes data and behavior into reusable objects.

Java is known for its Write Once, Run Anywhere principle. This means you can write your program once, and it can run on any device that has a Java Virtual Machine (JVM). Java is used in many areas like:

- **Web applications** (e.g., Amazon, eBay)
- **Mobile applications** (Android apps)
- **Desktop software** (e.g., Eclipse IDE)
- **Smart devices** (e.g., smart TVs, ATMs)

Why Java?

Java is a popular choice for developers for several simple reasons:

1. **Easy to Learn**: Java's rules are easy to understand, and its style is similar to other languages like C++. Once you know Java, learning other programming languages becomes easier.
2. **Works on Different Platforms**: Java programs can run on any device or operating system that has the Java Virtual Machine (JVM). This makes Java flexible, so you don't have to worry about what type of computer or system it's running on.
3. **Organized Code**: Java is an object-oriented language, which means it organizes code into classes and objects. This makes the code easier to manage, update, and reuse.
4. **Big Support Community**: Java has been around for over 20 years. There are many tutorials, forums, and resources available, making it easier to find help if you're stuck.
5. **Safe and Secure**: Java has built-in security features, making it a good choice for programs that need to be safe, like those involving internet connections or sensitive data.
6. **Lots of Ready-to-Use Tools**: Java offers many libraries and APIs (Application Programming Interfaces) that make tasks like working

with files, connecting to the internet, and managing databases simpler.

7. **Multitasking**: Java can handle multiple tasks at once. For instance, a program can keep receiving messages while you are using it for something else, like typing.

How Java is Platform Independent

Java is called platform-independent because its programs can run on different operating systems without any changes. Here's how it works:

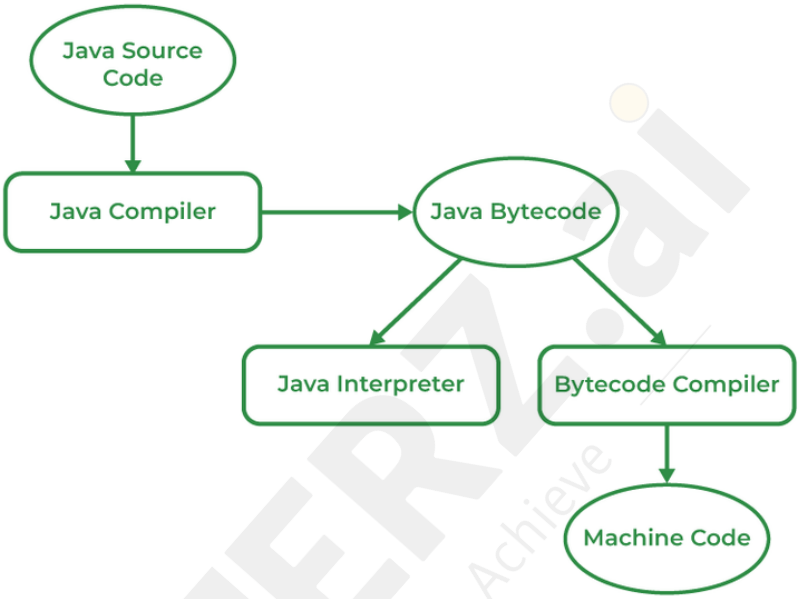
1. **Source Code**: A Java program starts with human-readable code, written by developers.
2. **Compilation**: The Java Compiler converts this source code into **Bytecode** (a type of code that is not specific to any operating system). This Bytecode is stored in a file with a `.class` extension.
3. **Java Bytecode**: This is not directly understood by the computer. However, it can be run on any device that has a **Java Virtual Machine (JVM)**.
4. **JVM (Java Virtual Machine)**: The JVM interprets the Bytecode and converts it into **Machine Code**, which is the language the computer can understand. Each operating system has its own JVM, making the JVM platform-dependent. But since the JVM translates Bytecode into Machine Code for the system it's running on, Java itself can run on any platform (like Windows, Mac, Linux, etc.).
5. **Platform Independence**: The reason Java is platform-independent is because the Bytecode can be executed on any operating system that has its own version of JVM. Since the JVM interprets the Bytecode, the same Java program can run anywhere, even though the JVM itself is different for each platform.

Important Points:

- **Java Bytecode**: Makes Java platform-independent.
- **JVM**: Translates Bytecode into machine language but is platform-dependent, meaning each operating system needs a different JVM.
- **Portability**: This feature of Java, called portability, allows you to run Java programs on any system that has the right JVM installed.

Step-by-Step Java Execution:

1. **Write the Program**: Developers write the Java program in human-readable code.
2. **Compile the Program**: The Java compiler converts the program into **Bytecode**.
3. **Run the Program**: The JVM interprets the Bytecode and runs the program on your machine.



By using JVM, Java can run on any machine, but the JVM itself has to be tailored for the specific operating system. This is why Java is platform-independent, but the JVM is platform-dependent.

JDK Installation

What is JDK?:

It's a toolkit that contains everything needed to develop and run Java programs

Installation Steps:

Simple steps to download and install JDK (with screenshots and links to official guides).

Visit the official website of oracle (**www.oracle.com**)

1. Click on the "Resources Tab"

Products Industries Resources Customers Partners Developers Company

2. Click on “Java Downloads”

Downloads

Customer Downloads Developer Downloads Java Downloads

3. Click on any version like Java 17

JDK 23 JDK 21 JDK 17 GraalVM for JDK 23 GraalVM for JDK 21 GraalVM for JDK 17

4. Select type of Operating System

Linux **macOS** **Windows**

5. Click on “Compressed Archive” and the JDK setup folder will be downloaded

x64 Compressed Archive	228.76 MB	https://download.oracle.com/java/23/latest/jdk-23_windows-x64_bin.zip (sha256)

6. **Checking Version**: Show code for checking Java version in command prompt

   Example: Command prompt code: **java - version**

   After entering java -version command below version should come

```
C:\Users\sukum\Downloads>java -version
java version "17.0.5" 2022-10-18 LTS
Java(TM) SE Runtime Environment (build 17.0.5+9-LTS-191)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.5+9-LTS-191, mixed mode, sharing)
```

7. **Setting Path**: Explain how to configure the PATH variable so Java can be run from the command line.

   Command line code for setting the path:

```
Microsoft Windows [Version 10.0.22631.4169]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sukum>set classpath=C:\Program Files\Java\jdk-17.0.5\bin

C:\Users\sukum>
```

Command is **set classpath=C:\Program Files\Java\jdk-17.0.5\bin**

To check the path we can use
C:\Users\sukum> echo %classpath%

After entering the above command we'll get the below output

```
C:\Users\sukum>echo %classpath%
C:\Program Files\Java\jdk-17.0.5\bin
```

What is JDK, JRE, JVM

1. JDK (Java Development Kit):

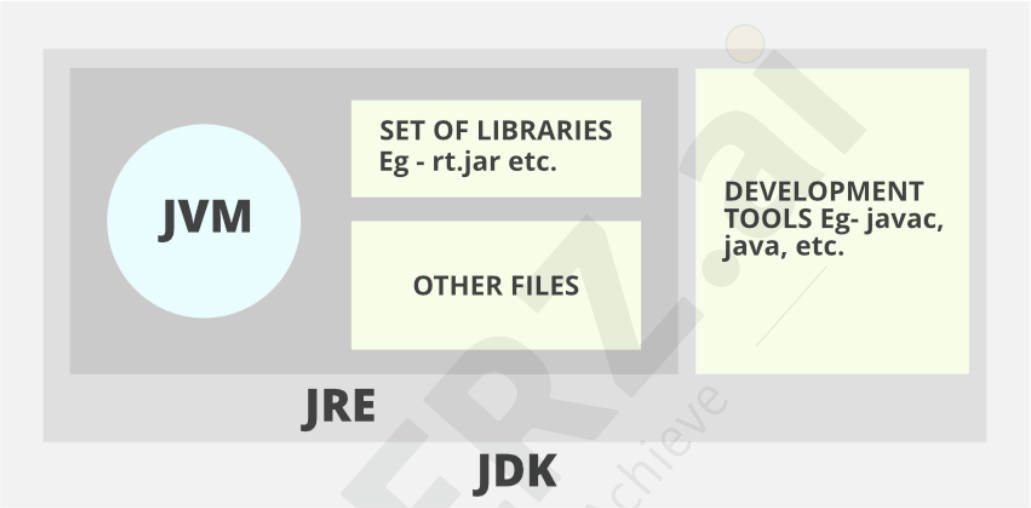
- The **JDK** is a complete toolkit that you need to **create** and **run** Java programs. It includes everything needed to write, compile, and run Java code.
- **What's inside the JDK?**
  - **Development tools** like the Java compiler (which turns your code into something the computer can understand).
  - **JRE (Java Runtime Environment)**, which is necessary to run Java programs.

2. JRE (Java Runtime Environment):

- The **JRE** is used to **run** Java programs. It doesn't help you to create programs, but it's necessary for running them.
- **What's inside the JRE?**
  - **JVM (Java Virtual Machine)**: This is the part of JRE that actually runs your Java program.
  - **Core libraries**: These are basic tools and files needed for running Java applications.
- **Who uses the JRE?** People who just want to run Java programs (not create them) use the JRE.

3. JVM (Java Virtual Machine):

- The **JVM** is the engine that **runs your Java program**. When you run any Java program, it gets executed inside the JVM.
- The JVM reads and executes your program line by line, making sure it works properly. This process is why JVM is often called an interpreter.
- The **JVM** is included in both the JDK (for developers) and the JRE (for users who want to run the program).



How They Work Together:

- **JDK** helps you write, compile, and run Java programs. It has everything a developer needs.
- **JRE** is used to run Java programs but doesn't include tools for writing or compiling them.
- **JVM** is inside both JDK and JRE. It's what actually runs your Java program on your computer.

In Simple Words:

- **JDK** is for creating and running Java programs.
- **JRE** is just for running Java programs.
- **JVM** is the part of JDK and JRE that makes your Java program work by reading it line by line.

Example:

Think of the **JDK** as a **toolbox** for building and running programs, the **JRE** as a **tool** to only run the program, and the **JVM** as the **worker** who actually does the job of executing the program.

This helps to understand that:

- Developers use **JDK** to create and run programs.
- Users only need **JRE** to run programs.
- **JVM** runs the program on any system that has JRE.

Java Program Execution

These are the two things that happen during the program execution.

- **Compilation**: Java code is written in a .java file and compiled into bytecode (.class file) using the JDK.
- **Execution**: The JVM runs the bytecode on any platform.

Step-by-Step: Running a "Hello, World!" Program

1. **Write the Program**:
    - Open **Notepad** (or any text editor).
    - Type this code in the file:

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

    - This program will print "Hello, World!" when run.
2. **Save the File:**
    - In Notepad, click File > Save As.
    - Name the file HelloWorld.java (make sure it ends with .java).
    - Under "Save as type," choose All Files.
    - Save it in a folder you can easily find, like C:\JavaPrograms.

3. **Open the Command Prompt:**
   - Press **Windows Key + R**, type cmd, and press Enter to open the command prompt.
   - Use the command below to go to the folder where you saved your Java file (for example, C:\JavaPrograms):

```
cd C:\JavaPrograms
```

4. **Compile the Program:**
   - In the command prompt, type this command to compile your Java file:

```
javac HelloWorld.java
```

5. **Run the Program:**
   - After compiling, you can run the program by typing:

```
java HelloWorld
```

   - This will run the compiled bytecode, and you will see the output:

```
Hello, World!
```

Simple Explanation:

- **Writing**: You write your code in a .java file (in our case, HelloWorld.java).
- **Compiling**: The command javac HelloWorld.java converts the Java code into **bytecode** and creates a .class file (in this case, HelloWorld.class).
- **Running**: The command java HelloWorld runs the bytecode on the **Java Virtual Machine (JVM)**, and you see the result on the screen ("Hello, World!").

Example Recap:

1. **Write**: HelloWorld.java
2. **Compile**: Creates HelloWorld.class (bytecode).
3. **Run**: Displays Hello, World! as output.

Installing Eclipse

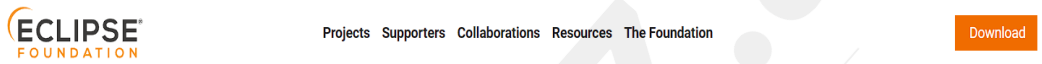
Why Eclipse?:

Eclipse makes easy writing, debugging, and running Java programs easy with a user-friendly interface.

Installation Steps: Guide learners to download and install Eclipse.

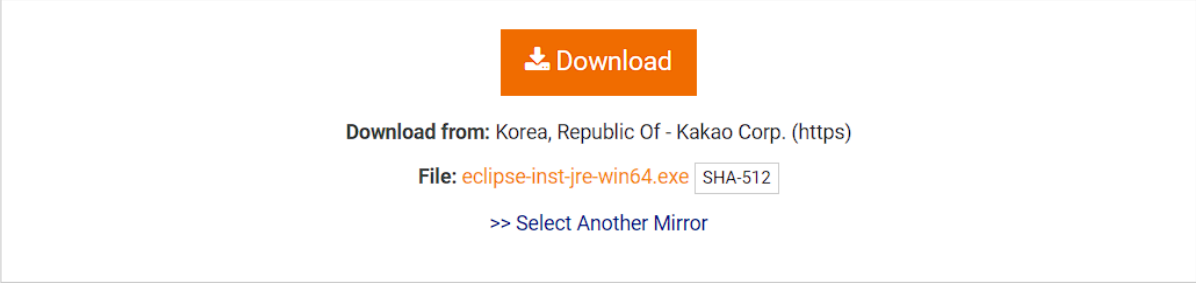
Example:

1. Visit Eclipse official website (www.eclipse.org)
2. Click on the download button



3. Select download

4. Select download



Basic Setup: Show how to create a simple Java project, write code, and run it.

Example: Create a simple project in Eclipse to display "Hello, I am Eclipse!" on the screen.

```Java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, I am Eclipse!");
    }
}
```

What is Static and Dynamic Typing?

Static / Strongly Typing (Java):

In static typing, you declare the type of a variable when you create it, like specifying that a variable will hold numbers, text, etc.

- **Example:**

```Java
int i = 10;
```

Here, int means the variable i is an integer, and it can only hold whole numbers.

- **Type Checking**: The computer checks the type before the program runs (at compile time).
- **Why Use Static Typing:** It catches errors early, makes your code more predictable, and can make programs run faster.

Dynamic Typing (Languages like Python):

In dynamic typing, you don't need to declare the type of a variable. The computer figures out what type it is based on the value you assign.

Example (in Python)

```
Java
i = 10;
```

Type Checking: The computer checks the type **while the program is running** (at runtime).

Why Static Typing (like in Java) Is Useful:

- **Fewer Errors**: Since the type is checked before the program runs, you can catch mistakes early.
- **Predictable Code**: You know exactly what type of data each variable will hold, which makes the code easier to understand.
- **Better Performance**: Static typing can make programs run faster because the computer knows the types ahead of time.

Variables and Data Types

Java Data Types

In Java, everything you store in a program—like numbers, text, or true/false values—needs to be assigned a specific data type. Java is a statically typed language, meaning you must declare the type of any variable when you write the code. It's also strongly typed, which means once a variable is assigned a type, it cannot be used as a different type unless explicitly converted.

There are two main categories of data types in Java:

1. **Primitive Data Types**: These are the most basic types in Java, and they store simple values.
2. **Non-Primitive Data Types**: These are more complex types like objects, arrays, and strings.

Primitive Data Types

Java has **8 primitive data types**. Let's break them down:

Type	What It Stores	Example Value	Size	Range
boolean	true or false	true, false	1 byte	Only true/false
byte	Small integers	10, -50	1 byte	-128 to 127
char	A single character	'A', '%'	2 bytes	0 to 65,535 (Unicode values)
short	Small whole numbers	-150, 3000	2 bytes	-32,768 to 32,767
int	Standard whole numbers	42, -999	4 bytes	-2,147,483,648 to 2,147,483,647
long	Large whole numbers	100000L	8 bytes	-9 quintillion to 9 quintillion
float	Decimal numbers (less precise)	3.14f	4 bytes	Up to 7 decimal places
double	Decimal numbers (more precise)	3.14159d	8 bytes	Up to 16 decimal places

Explanation of Each Primitive Data Type:

1. **boolean**:
   - Stores either true or false. Useful for conditions and decisions in programs.
   - Example:
     boolean isStudent = true;

2. **byte:**
   - Stores small whole numbers. Often used in large arrays where saving memory is important.
   - Example:
     byte age = 25;
3. **char:**
   - Stores a single character, like a letter or a symbol, using Unicode values.
   - Example:
     char letter = 'A';
4. **short:**
   - Like int, but for smaller whole numbers. It's also used to save memory in some cases.
   - Example:
     short distance = 1200;
5. **int:**
   - The most commonly used data type for whole numbers.
   - Example:
     int number = 100;
6. **long:**
   - Used for very large whole numbers when int is not enough.
   - Example:
     long population = 7000000000L;
7. **float:**
   - Stores decimal numbers, but with less precision than double. Useful when you want to save memory.
   - Example:
     float price = 9.99f;
8. **double:**
   - Stores decimal numbers with higher precision than float. It's commonly used for precise calculations.
   - Example:
     double pi = 3.14159265359;

In many programming languages, like **C/C++**, a `char` (which represents a character) is **1 byte** (or **8 bits**) in size. This is enough to store **ASCII** characters, which include basic letters, numbers, and symbols. ASCII can handle only **256 characters**, which works well for English but not for other languages.

Why does Java use 2 bytes for `char`?

Java uses a system called **Unicode** instead of ASCII. Unicode can represent almost all written languages, including English, Spanish, Chinese, Arabic, and many more. Since Unicode has a lot more characters than ASCII, **1 byte** (8 bits) isn't enough.

That's why Java uses **2 bytes** (or **16 bits**) for each `char`. This allows Java to store **65,536 different characters**! This means Java can handle characters from many languages, making it more versatile.

- **C/C++**: `char` is **1 byte** and can store 256 characters (ASCII).
- **Java**: `char` is **2 bytes** and can store 65,536 characters (Unicode).

Non-Primitive (Reference) Data Types

In Java, **Non-Primitive (Reference) Data Types** do not store the actual value directly in memory. Instead, they store a reference (or memory address) to where the value is located. Some common non-primitive data types are **Strings**, **Classes**, **Objects**, **Interfaces**, and **Arrays**.

AIMERZ.ai
Aim.Act.Achieve

THANK YOU

AIMERZ.ai

Aim.Act.Achieve